

# Spécifications Techniques

## CAHIER DES CHARGES — API REST RUBY ON RAILS

VERSION 1.2

PROJET  
API TaskManager

TECHNOLOGIE  
Ruby on Rails 7.1

BASE DE DONNÉES  
PostgreSQL / SQLite

AUTEUR  
Équipe Backend

### 1. PRÉSENTATION & OBJECTIFS DU PROJET

Ce cahier des charges définit les spécifications pour la création d'une API backend centralisée construite en architecture **RESTful** sous **Ruby on Rails** (mode `--api`). L'objectif est d'offrir une interface de gestion fluide et normalisée pour manipuler les données d'utilisateurs et de leurs tâches respectives.

### 2. ARCHITECTURE & MODÉLISATION TECHNOLOGIQUE

#### Socle Ruby on Rails

CORE

- Ruby 3.2+** : Langage backend principal.
- Rails 7.1 API** : Architecture allégée sans la vue HTML pour un maximum de performances.
- Puma** : Serveur d'applications multi-thread.

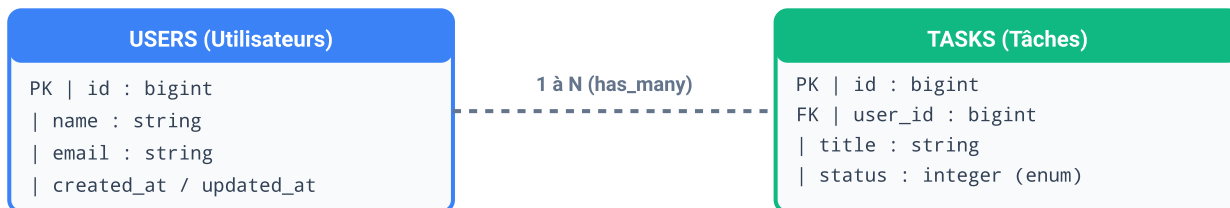
#### Gestion des Données

DATA

- PostgreSQL** : SGBD relationnel robuste.
- Active Record** : ORM pour les associations, validations et requêtes SQL.
- JSON Serializer** : Standardisation du format des réponses.

### 3. DIAGRAMME RELATIONNEL DES DONNÉES (ERD)

La base de données repose sur une relation directe **1 à N (has\_many)** entre les utilisateurs et les tâches.



## 4. SPÉCIFICATION DES ROUTES API REST

L'API met à disposition l'ensemble des routes CRUD pour la ressource Task ainsi qu'un accès aux tâches filtrées par utilisateur via la route User / Task.

| MÉTHODE | ENDPOINT                     | DESCRIPTION / ACTION                                                       |
|---------|------------------------------|----------------------------------------------------------------------------|
| GET     | /api/v1/tasks                | Récupérer <b>toutes</b> les tâches (Tasks => all)                          |
| GET     | /api/v1/tasks/:id            | Consulter une tâche spécifique par son ID (get/:id)                        |
| POST    | /api/v1/tasks                | Créer une nouvelle tâche                                                   |
| PUT     | /api/v1/tasks/:id            | Remplacer/Mettre à jour intégralement une tâche                            |
| PATCH   | /api/v1/tasks/:id            | Mettre à jour partiellement une tâche (ex: statut)                         |
| DELETE  | /api/v1/tasks/:id            | Supprimer une tâche par son ID                                             |
| GET     | /api/v1/users/:user_id/tasks | <b>User/Task</b> : Obtenir uniquement les tâches associées à l'utilisateur |

## 5. STRUCTURE DES RÉPONSES (FORMAT JSON)

Exemple : GET /api/v1/users/5/tasks

HTTP 200 OK

```
[ { "id": 101, "title": "Développer l'API REST Ruby on Rails", "status": "in_progress", "user_id": 5, "created_at": "2026-10-06T10:00:00Z" } ]
```

## 6. AMÉLIORATIONS & ÉVOLUTIONS FUTURES

Sécurité & Authentification

SECURITÉ

- Authentification JWT** : Sécurisation future des endpoints via jetons JSON Web Tokens (Devise-JWT).
- Gestion des Rôles** : Contrôle d'accès pour s'assurer qu'un utilisateur ne modifie que ses propres tâches.

Optimisation & Outillage

TECH OPS

- Pagination & Filtres** : Intégration du système de pagination (gem Pagy).
- Documentation Swagger** : Auto-génération de la documentation via rswag pour faciliter l'intégration.